

Before/At the start of the workshop

Download and install [MPLAB X IDE v6.05](#) (Windows)

- Also available for:
 - [Mac](#)
 - [Linux](#)
- NOTE: Support for the PIC Kit 3 was dropped in later versions of MPLAB X IDE, so you'll need to use this *slightly* older version for the workshop.

Download and install [XC8](#)

- You should be promoted to do this at the end of installing MPLAB X IDE.

(Optional) Learn about how microcontrollers like the Arduino and PIC actually work

- [Introduction to Microcontrollers](#) (DeepBlueMbedded)
- [Beyond Arduino](#) (Nathan Jones)

During the workshop

(Loosely follows the [Instructables](#) for Hackerbox #0040.)

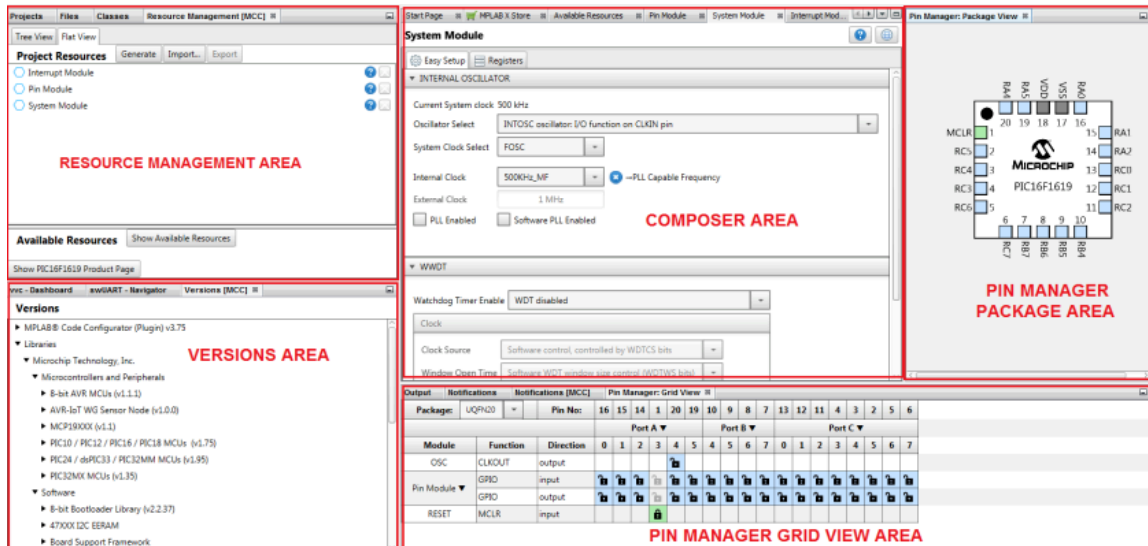
Prepare your kit

(Loosely corresponds to Steps 1 and 2 of the [Instructables](#).)

- Inventory your kit. You can find a parts list on the [product page](#) for the Hackerbox #0040.
- Ensure you have a PIC16F18446 microcontroller and USB breakout cable to use for the remainder of the workshop (they are separate from the Hackerbox #0040 kit).
- Now or at some point during the workshop, solder the following:
 - Pin header on the character LCD
 - Pin header to the GPS cable
 - (optional) Extra solder on the ground tabs on the USB mini B jack on the breadboard power supply. It breaks off *very* easily, otherwise; mine broke off first thing, which is why I ordered the USB breakout cables.

Start a project in MPLAB X for the PIC16F18446 and open MCC

- Follow the instructions [here](#) and then open the MPLAB Code Configurator () once complete.
- Identify what modules are available to you in MCC for programming this device; you can find the list in the “Device Resources” window in the “Resource Management” area.



For any you're not sure of, head to the [product page](#) for the PIC16F18446 and see what you can learn there.

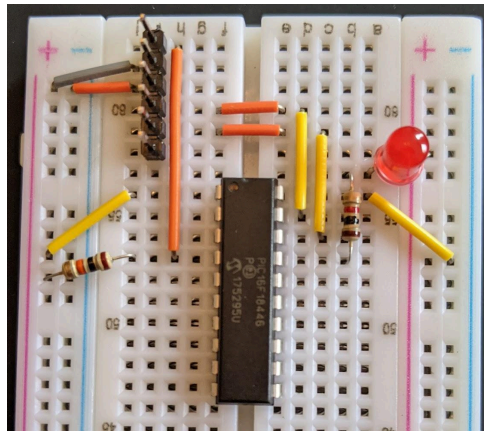
- Pick a few features or peripherals on the PIC16F18446 that seem interesting and write down a short description about how they work, or what they could be used for, by:
 - Reading the relevant portion of the datasheet
 - Reading the relevant portion(s) of the Microchip documentation (start [here](#) or [here](#))
 - Conducting an internet search for that feature/peripheral
 - Adding the peripheral () to your “Project Resources” and then inspecting what configuration fields are available in the “Composer Area”.
- Take note of the pinout in either the “Pin Manager Package Area” (you may need to change the package to “PDIP20”) or on page 7 of the datasheet. We'll be referring to it any time we build a circuit around the PIC16.
- Peruse the rest of the “Documentation” section on the [product page](#) for the PIC16F18446 and write down or save for later a few documents that seem interesting (such as the [PIC Microcontroller Compiled Tips 'n Tricks Guide](#)!).

Blink an LED

(Loosely corresponds to Steps 3-5 of the [Instructables](#).)

- Follow the instructions [here](#).

- You'll need to build the circuit yourself, since you're not using a development board.
 - Make the following connections to the PIC16:
 - Pin 01 / "VDD" → Breadboard power rail (5V)
 - Pin 04 / "nMCLR/VPP/RA3" → PIC kit 3, Pin 1
 - Pin 18 / "RA1/ICSPCLK" → PIC kit 3, Pin 5
 - Pin 19 / "RA0/ICSPDAT" → PIC kit 3, Pin 4
 - Pin 20 / "VSS" → Breadboard ground rail
 - Provide power to your breadboard by:
 - Attaching the breadboard power supply and plugging in either a USB or DC jack cable (don't forget to add some solder to the USB jack if you're going to use that!).
 - Connecting pins 2 and 3 of the PIC kit 3 to the breadboard power rails and following this [guide](#) for using the PIC kit 3 to provide power.
 - The following components are recommended, but not strictly required:
 - 10 kOhm pull-up resistor on nMCLR
 - Filtering capacitors between VDD and VSS
 - Example



- Take note that "RA4" is used to drive the LED in that guide only because that's the pin that the LED is connected to on that specific development board; you should use whichever pin you connect your LED to.
- You can ignore the comment about enabling low voltage programming.
- When I built this circuit, MPLAB complained of an "unknown configuration setting/register" in reference to the DEBUG setting. The datasheet states that this bit is managed by the programming tools so I just commented out that line with no ill effect.
- You can also use the PICkit3 to program the PIC with the programming target (the board with the light blue ZIF socket); see the picture in Step 3 of the [Instructables](#). This would be most useful to program many PICs in a row with the same code.
- Identify the other functions that MCC created by looking through "Header Files/MCC Generated Files/pins.h".

Use breakpoints, single-stepping, and memory views in MPLAB X IDE to debug a program

- Watch the video [here](#) about using the debugger in MPLAB X.
- Follow along with the video above using the sample code below (change `IO_RB7_SetHIGH()` / `IO_RB7_SetLow()` to the correct macros that were generated for your project from above).

```
void delay(int val)
{
    for(int i = 0; i < val; i++);
}

int main(void)
{
    ...
    volatile int delay_val = 50;
    ...

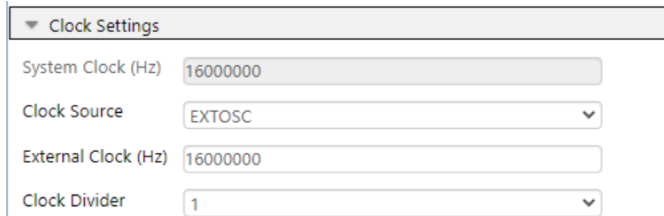
    while (1)
    {
        IO_RB7_SetHigh();
        delay(delay_val);
        IO_RB7_SetLow();
        delay(100-delay_val);
    }
}
```

- Change `delay_val` (instead of `ctr`) using the debugger to any value from [0, 100] and observe the effect on the LED.
- You can also control the peripheral registers directly from the debugger! In the “Variables” window, click “Add new watch” (🔍). Select “SFRs” and find LATZ, where Z is the port to which your LED is attached (A, B, C, etc). Expand the LATZ variable when it shows up and change the value of LATZ# (e.g. LATA1 if your LED is connected to RA1) from 1 to 0 and back again.
- Pause to revel in the utter awesomeness of being able to control nearly EVERY POSSIBLE ASPECT of your microcontroller! Your ability to find and fix errors in your code, to build cooler and more complex projects, has never been greater.
- You may also find these resources helpful:
 - [MPLAB X IDE: Debugging with Advanced Breakpoints](#) (Microchip)
 - [Rules of Debugging](#) (Embedded Artistry)
 - [How to Debug](#) (John Regehr)
 - [Six debugging techniques for embedded system development](#) (Control Engineering)

(Optional) Use an external oscillator to clock the PIC16F18446

(Loosely corresponds to Step 6 of the [Instructables](#).)

- Connect one of the 16 MHz crystal oscillators to pins 2 and 3 (OSC1 and OSC2; information located in Table 3 of the datasheet) of the PIC16F18446 and two capacitors of equal value between each pin and ground (see the schematic in the upper-right portion of the picture in Step 6 of the [Instructables](#)).
- Change the “Clock Control” resource to look like this:



The screenshot shows a configuration window titled "Clock Settings". It contains four fields: "System Clock (Hz)" with a value of 16000000, "Clock Source" set to "EXTOSC" via a dropdown menu, "External Clock (Hz)" with a value of 16000000, and "Clock Divider" set to 1 via a dropdown menu.

- If you run the same code as from the previous section, you’ll notice that the LED blinks much faster than before! Sixteen times faster, to be exact, since the clock frequency is now sixteen times greater than it was before.
- Of course, the internal oscillator that we’ve been using to clock the PIC can also be set as high as 32 MHz (change the “Clock source” back to “HFINTOSC” and see how changing the frequency and clock divider affect the “System Clock”), so speed isn’t necessarily the reason to use an external oscillator, but *precision* might be: the internal oscillator on the PIC is accurate to about $\pm 8\%$ whereas the crystal oscillator is probably accurate to about 50-100 ppm (that’s 0.0001%). However, you only get that accuracy (and you only avoid problems with crystal start-up and longevity) if the capacitor values are chosen correctly, which isn’t easy to do.
- Additional resources:
 - Microchip appnote AN826, “Crystal Oscillator Basics and Crystal Selection for rPIC and PICmicro Devices”
 - Microchip appnote AN849, “Basic PICmicro Oscillator Design”
 - Microchip appnote AN943, “Practical PICmicro Oscillator Analysis and Design”
 - Microchip appnote AN949, “Making Your Oscillator Work”

Display a message on the Character LCD

(Loosely corresponds to Step 7 of the [Instructables](#).)

- Read or skim the first four sections of [this LCD guide](#) (“Introduction” through “Hardware Assembly”).
 - Solder the header pins onto the LCD module if you haven’t already.
- Connect the LCD to power (two pins), ground (three pins), a potentiometer to control the contrast, and to six GPIO pins on the PIC16.
 - Use either the schematic in Step 7 of the [Instructables](#) or the “Hardware Hookup” section of the above guide for help.

- In MCC, make six outputs (corresponding to the six pins that you connected to your LCD) and change the custom names to “RS”, “EN”, “LCD_D4”, “LCD_D5”, “LCD_D6”, and “LCD_D7”. Double-check that the names correctly correspond to those same pins on the LCD!
- Paste the [LCD functions](#) into your `main.c` file above the `main` function.
- Put the following above your `while(1)` loop and run the code:

```
lcd_init();
lcd_clear();
lcd_setCursor(1,1);
lcd_puts("Hello world!");
lcd_setCursor(2,1);
lcd_puts("LCD operational");
```

Build a clock

(Loosely corresponds to Step 8 of the [Instructables](#) and [this GPS project](#).)

- The GPS antenna sends out NMEA messages over UART, some of which contain information about the current UTC time and date. See [here](#) for a brief discussion of what those messages look like. See [here](#) for a description of what the UART protocol is.
- Solder the GPS antenna cable to a 6-pin header, if you haven't already, and then make the following connections:
 - Pin 2 / VCC → 5V
 - Pin 3 / TXD → Any free GPIO on the PIC
 - Pin 5 / GND → Ground
- Add a UART driver to your project in MCC and configure it like below:

The screenshot shows the 'Configuration Settings' window for a UART driver in Microcontroller Configuration (MCC). The settings are as follows:

Setting	Value
Custom Name	UART1
Requested Baudrate	9600
Calculated Baudrate	9592
Baud Rate Error (%)	0.08
Parity	None
Data Size	8
Stop Bits	1
Flow Control Mode	None
Redirect Printf to UART	Ensure Redirect to Printf is enabled for only one UART driver. (Toggle is OFF)

- In the “Pin Grid View”, make sure that the RX pin correctly corresponds to the pin to which you connect the “TXD” pin of the GPS antenna cable.
- Let's start by just looking at what messages are being sent by the GPS antenna.
 - Near the top of your main file, add this: `#define BUFFER_LEN 128`

- Then in your `while(1)` loop, add the following code:

```
static char buffer[BUFFER_LEN];

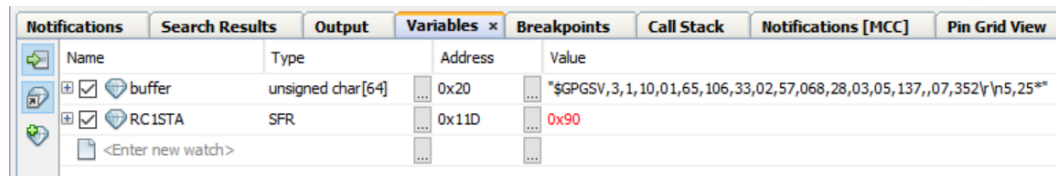
memset(buffer, 0, BUFFER_LEN);

// Find start of next message
while( buffer[0] != '$' )
{
    while( !UART1__IsRxReady() );
    buffer[0] = UART1_Read();
}

// Fill in the rest of the message
int pos = 0;
while( pos < BUFFER_LEN && buffer[pos] != '\n' )
{
    while( !UART1__IsRxReady() );
    buffer[++pos] = UART1_Read();
}

NOP();
```

- Start a debug session and set a breakpoint on the line that says `NOP()`, then wait until the breakpoint is reached. You may need to move closer to a window to help the GPS antenna get a good signal.
- Once the breakpoint is reached, select your “Variables” window and find `buffer`. It should look something like the image below. If not, right-click on `buffer` and select “New watch” (or something like that).



The screenshot shows the 'Variables' window in an IDE. It has tabs for 'Notifications', 'Search Results', 'Output', 'Variables', 'Breakpoints', 'Call Stack', 'Notifications [MCC]', and 'Pin Grid View'. The 'Variables' tab is active, displaying a table with columns: Name, Type, Address, and Value. There are three entries: 'buffer' (type 'unsigned char[64]', address '0x20', value '\$GPGSV,3,1,10,01,65,106,33,02,57,068,28,03,05,137,,07,352\r\n5,25*'), 'RC1STA' (type 'SFR', address '0x11D', value '0x90'), and '<Enter new watch>' (type '...', address '...', value '...').

Name	Type	Address	Value
buffer	unsigned char[64]	0x20	\$GPGSV,3,1,10,01,65,106,33,02,57,068,28,03,05,137,,07,352\r\n5,25*
RC1STA	SFR	0x11D	0x90
<Enter new watch>	...	...	...

- Click the “continue” button several more times to see different GPS messages. Use the resource above to identify the contents of some of the messages.
- Now we’re going to try to find the message type called RMC that contains time and date information.
 - Add `#include "string.h"` near the top of your main file.
 - Change the line that says `NOP()` to the following:

```
// Determine if this is an RMC message
if( strstr(buffer, "RMC") )
{
    NOP();
}
```

- Start another debug session and put a breakpoint again on the `NOP()`.
 - When the breakpoint is reached, inspect `buffer` again. Identify the fields of the RMC message, including the time and date. Are the time and date correct? (The time is in UTC.) What about the lat/long coordinates? Can you paste them into a mapping program to find our location?
- Lastly, we're going to parse the RMC message to get the UTC time and date and display it on the LCD.
 - Replace the `if` statement with the code found here: [Parsing RMC messages](#)
 - You can find a copy of the full `main.c` file I used here: [GPS clock main file](#)
 - Run the code and (hopefully) observe your GPS clock!
- The time given by the GPS message is UTC (Greenwich Mean Time). Modify the code to display the current time for your hometown by adding/subtracting an offset (don't forget to handle wraparound!). You may need to learn a little about the [24-hour clock](#), since that's the format that UTC is in.
- Identify a few other ways that the sample code could be modified or improved.

At the end of/after the workshop

Extend the clock

- Add an alarm
- Add power saving by putting the PIC16 to sleep periodically or reducing how frequently the GPS is used
- Add buttons to set the time/time zone
- Handle daylight savings
- Build an enclosure
- Show lat/long on the LCD (at a set frequency [such as one second for the time/date and one second for the position], using a button to toggle which is on the screen, etc).

Learn how to use a new peripheral

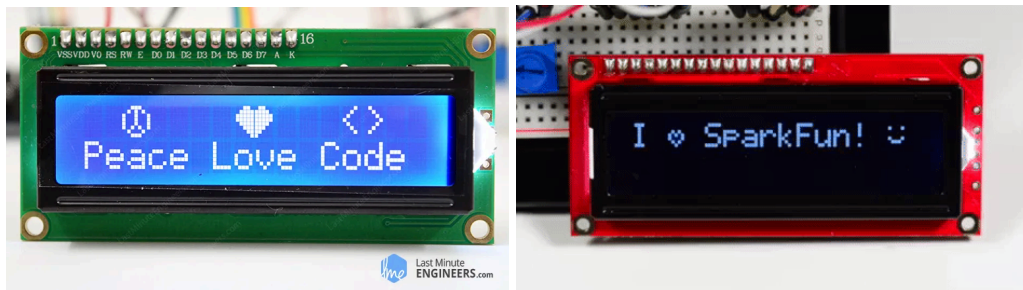
- Interrupts, timers, PWM, I2C, SPI, and ADC are fundamental ones to learn.
- The PIC16F18446 also has lots of unique peripherals such as:
 - Configurable logic cell (CLC)
 - Complementary waveform generator (CWG)
 - Numerically controlled oscillator (NCO)
 - Data signal modulator (DSM)
 - Peripheral pin select (PPS)
 - Signal measurement timer (SMT)
 - Zero-crossing detect (ZCD)
 - Voltage references

LED projects

- Drive the LED using:
 - Non-blocking code, like in this [Arduino example](#) (you'll need to use a timer to implement a "millis()" function by either (1) causing the timer to increment every millisecond or (2) causing the timer to overflow every millisecond and then incrementing a software variable)
 - A timer interrupt
 - The CCP module
- Create a fading/breathing effect (slowly brightening/dimming)
- Use buttons to control the brightness/fading
- Use UART to control the brightness/fading

Character LCD projects

- Read through the sample code shown for the LCD and describe how it works. You may find these additional resources helpful:
 - [How to Use Character LCD Module](#) (elm-chang.org)
 - [Interfacing 16x2 LCD With PIC Microcontrollers](#) (DeepBlueMbedded)
- Add or modify one thing in the sample code to test out an additional feature or use of the character LCD, such as scrolling the on-screen text, blinking the cursor, or making a custom character.
- Port [Hunt the Wumpus](#) from Arduino to the PIC
- Display a word/quote of the day
- Build the [Endless Runner](#)
- Read about making custom LCD graphics [here](#) and then use [this tool](#) to make or use some of your own. Display custom messages like the ones below:



(Note that in the image to the right the heart is alternately filled and unfilled.)

UART projects

- Connect your PIC to a USB-to-UART adapter ([like this one](#)) instead of the GPS antenna and then send/receive messages from your computer. You might find [this reference](#) helpful; the final project there shows an exceedingly simple command-line interface (CLI) to a PIC that is used to control two LEDs (see [this reference](#) for some examples of how to turn that into a much more capable CLI).

GPS projects

- Build a [reverse geocache](#)
 - In lieu of a locked box, you could just print a message to the LCD when the GPS is within a certain range of a point.
 - Modify the reverse geocache to make a device that helps you find your car in the middle of a large parking lot.
- Build a [speedometer](#) or a device that tells you how far you've walked in a day.
- The GPS module has an RX pin for a reason! You can actually send it commands to modify things like power settings or which messages get sent out the TX pin. Find the datasheet for the GPS module (a datasheet for a very similar GPS can be found [here](#)) and play around with sending it commands.

Find a [new PIC](#), pick up a [SNAP debugger](#), and program it using the [Xpress online IDE](#)

- The Xpress online IDE has a slightly simpler interface and some folks might prefer it to the desktop IDE, MPLAB X.
- However, newer versions of MPLAB X (including the Xpress online IDE) don't support the PICkit3. You'll need to pick up either the [SNAP debugger](#), [PICkit4](#), or [PICkit5](#).

Learn advanced debugging techniques

- Learn how to use the [simulator](#) in MPLAB X
- Learn how to set [data](#), [event](#), and complex breakpoints
 - There are two types of complex breakpoints: [sequence](#) and [tuple](#)

Program the two PICs that came in Hackerbox #0040 (PIC12F675 and PIC16F628A)

- Unfortunately, neither of those microcontrollers are supported by MCC (so you'll have to configure the peripheral registers directly) and you can't debug them without an additional (obsolete) piece of hardware. But you can learn to program them in two ways:
 - Locate the datasheets and follow along with one of these tutorials:
 - [PIC Basics](#) (DeepBlueMbedded)
 - [PICGuides.com](#)
 - Look more closely at the "MCC Generated Files" and observe how many of the convenience functions we were using today (such as `LED_SetHigh()`) are simply reading from or writing to a peripheral register. Replace the function calls with the register accesses and convince yourself that the program still works. Then start a new project for the two PICs that came in the kit and copy over the code, using the datasheet and the above tutorials to make sure all of the registers are still named the same.

- You can still debug via `printf` (once you set up the UART) or by toggling a GPIO pin, just not by setting breakpoints and the like.